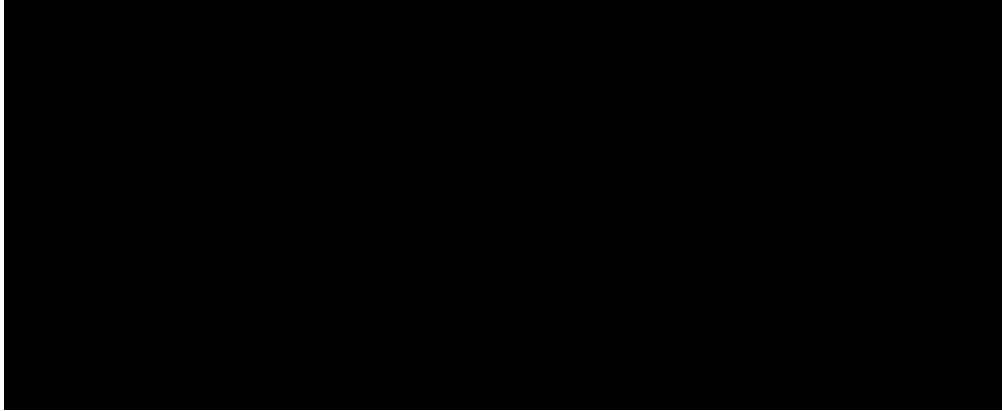
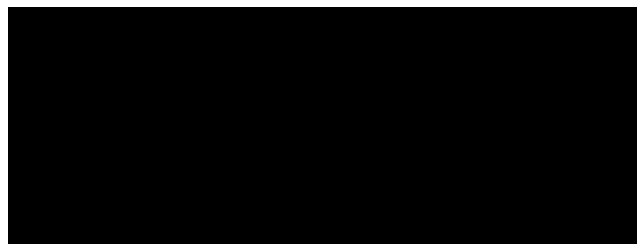




A DISSERTATION SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF MASTER OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING

2025



Department of Computer Science

Contents

Abstract	8
Declaration	9
Copyright	10
Acknowledgements	11
1 Introduction	12
1.1 Motivation	12
1.2 Aims and Objectives	13
1.3 Background Concepts	14
1.3.1 Handheld Radios	14
1.3.1.1 Radio Licensing	15
1.3.2 Fast Fourier Transform	17
1.3.2.1 Frequency Modulation and Audio Signal Encoding	18
1.3.3 Cyber Security Principles	19
1.3.4 Mobile Phone Development	20
1.3.5 Android App Structure	20
1.4 Ethical Considerations	21
2 Methodology	23
2.1 Android App Framework	23
2.2 Sending Messages	24
2.2.1 Text-to-Audio Encoding	24
2.2.2 Audio Playback	25
2.3 Receiving Messages	26
2.3.1 Timing Synchronisation	26

2.3.2	Audio-to-Text Decoding	27
2.3.2.1	TarsosDSP	28
2.4	Encryption	29
2.4.1	Contacts	32
2.5	Handheld Radios	33
3	Evaluation and Reflection	34
3.1	Test Design	34
3.2	Test Results	37
3.2.1	Message Length and Encryption	38
3.2.2	Noise	40
3.2.3	Range	43
3.3	Critical Reflection Outside Data	44
4	Conclusion	45
4.1	Summary	45
4.2	Limitations and Further Work	46
4.2.1	UI	46
4.2.2	Testing	46
4.2.2.1	Test Repetition	46
4.2.2.2	Conditions	47
4.2.2.3	Hardware	47
4.2.3	Noise Resilience	47
4.2.3.1	Forward Error Correction (FEC)	48
4.2.3.2	Adaptive Modulation	48
4.2.3.3	Signal Averaging FFT with Moving Window	49
	Bibliography	50
A	Software Versions	57
A.1	Android Studio	57
A.2	Build Dependencies	57
A.3	Phones used for testing	58
A.3.1	Sender	58
A.3.2	Receiver	58

B System Parameters

59

Word Count: 8922

List of Tables

3.1	Messages, their lengths, and re-sync chords required.	34
3.2	The test conditions.	37
3.3	Message success rates across ranges, encryption modes, noise profiles, and total messages received.	38
3.4	Total messages sent and received, and success rate over all tests. . . .	38
3.5	Message lengths and success rates, regardless of encryption.	39
3.6	Message lengths and success rates for unencrypted and encrypted mes- sages.	41
3.7	Success rates for encrypted and unencrypted messages across all tests.	41
3.8	Success rates for encrypted and unencrypted messages under ideal conditions.	42
3.9	Noise levels and total messages received	42
3.10	Noise levels and total messages received; messages split into unen- cryptured (UE) and encrypted (E).	43
3.11	Ranges and total messages received	44
3.12	Ranges and total messages received <i>under noisy conditions only</i> . . .	44

List of Figures

1.1	The message transmission pipeline.	14
1.2	Block diagram of a radio transmitter's internal components.	16
1.3	The effect of Frequency Modulation on a carrier wave.	16
1.4	The effect of the DFT (and FFT).	17
1.5	Confirmation from the University of Manchester Ethical Approval Tool.	22
2.1	The sending and receiving messages page.	29
2.2	The timing mechanism between the sender and receiver.	30
2.3	The AES-GCM decryption and authentication process.	31
2.4	The contacts page and the page for entering a new contact.	33
3.1	The frequencies present in the selected portion of <i>Contact</i> by <i>Daft Punk</i>	36
3.2	How success rate decreases as plaintext message length increases.	39
3.3	Success rate as message length increases and encryption is applied.	41

List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
BER	Bit Error Rate
CTR	Counter
DFT	Discrete Fourier Transform
FCC	Federal Communications Commission
FEC	Forward Error Correction
FFT	Fast Fourier Transform
FSK	Frequency Shift Keying
GCM	Galois/Counter Mode
Hz	Hertz
IDE	Integrated Development Environment
IV	Initial Value
JIT	Just-In-Time
PCM	Pulse-Code Modulation
PTT	Push-To-Talk
RF	Radio Frequency
RSGB	Radio Society of Great Britain
SDK	Software Development Kit
SNR	Signal-to-Noise Ratio
UHF	Ultra-High Frequency
UI	User Interface
VHF	Very-High Frequency
VOX	Voice Operated Exchange
W	Watt
XML	Extensible Markup Language

Abstract

This project explores the development of a secure, infrastructure-independent communication system by combining handheld radios with an offline smartphone application. Motivated by the growing importance of privacy and the limitations of mobile phone networks – particularly in areas lacking infrastructure or during outages – the system encodes text messages into audio, transmits them over using handheld radios, and decodes them back into text using an Android application, “*Nuntius*”. The app incorporates real-time frequency-shift keying (FSK)-based modulation and the Fast Fourier Transform (FFT), alongside optional encryption through the Advanced Encryption Standard in Galois Counter Mode (AES-GCM) to ensure message confidentiality, integrity, and authenticity.

Testing was conducted under varied conditions, including differences in range, noise environments, message length, and encryption. Results demonstrated the feasibility of the system, with an overall success rate of 58.9% which increased to 84.68% in quiet environments. Significant vulnerability to noise was highlighted, with rates falling to 28.96% under high-noise conditions. These findings underline both the promise of the approach and the need for further refinement.

Future directions include implementing forward error correction (FEC), adaptive modulation, and FFT averaging to improve resilience, expanding testing across environments and hardware, and enhancing the app’s user interface. Overall, the project demonstrated that radios and smartphones can be effectively integrated into a practical, secure, and low-cost communication solution, while also mapping out key challenges for real-world deployment.

Declaration

No portion of the work referred to in this dissertation has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

- i. The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- ii. Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.
- iii. The ownership of certain Copyright, patents, designs, trade marks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the thesis, for example graphs and tables (“Reproductions”), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- iv. Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see <http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420>), in any relevant Thesis restriction declarations deposited in the University Library, The University Library’s regulations (see <http://www.library.manchester.ac.uk/about/regulations/>) and in The University’s policy on presentation of Theses

Acknowledgements

I would like to thank my supervisor, [REDACTED] for his support and invaluable guidance throughout this rewarding project.

My gratitude also goes to my loved ones for their emotional support and proof reading skills, without which this project would not have been possible.

Chapter 1

Introduction

1.1 Motivation

Secure, private communication is universally regarded as a fundamental human right in contemporary society. One of the most poignant examples of the numerous and widespread protections of privacy internationally is Article 12 of the UN Universal Declaration of Human Rights: “No one shall be subjected to arbitrary interference with his privacy, ..., or correspondence” [69]. In addition, the Geneva Convention IV – Protection of Civilians [35] and the Just War Theory principle of Jus In Bello (“Right Conduct in War”) [18] can be applied to the protection of privacy internationally. Concurrently, these demonstrate the legal and moral framework within which secure communication occurs.

Despite the universal acknowledgement of the right to privacy, numerous states routinely violate these protections in practice. Weak legal frameworks, mass surveillance laws, and opaque enforcement mechanisms continue to erode privacy globally [38]. This discrepancy reveals a critical gap between normative values and actual state behaviour, underscoring the necessity of projects such as this that develop and advocate for effective protections.

In the Information Era, mobile phones are one of the most ubiquitous means of communication across the world, with 4.6 billion people (57% of the population) using mobile internet in 2024 [23]. This technology allows easy, instantaneous communication worldwide. In junction, free applications such as WhatsApp and Telegram can ensure that these communications are secure.

This interconnectivity relies on a large amount of infrastructure. Mobile phones require a cellular network to function; this includes base stations, antennas, and a core

network. These base stations – also known as cell sites – handle wireless communication within a specific area, and the core network manages data routing and mobile communications. When any one element of the infrastructure fails, secure communication comes to a complete halt. This fact, as well as the billions of people without any mobile internet access [23] present an alarming situation, where effective communication is not possible, private or otherwise.

The circumstances of the communication breakdowns or absences should also be considered. They may be due to the individual's circumstances, for example living in a remote location, lack of infrastructure, poverty, or it could come in times of crisis. The crisis could be due to many different reasons, such as natural disasters, weather events, or conflict situations. In all such crisis situations, whatever the cause, effective private communications are of vital importance. An example of such a crisis is the ongoing war in Ukraine, where cyber-attacks targeting the military have also affected personal and commercial users [16].

This project aims to tackle these troubling scenarios by presenting a method of secure communication that does not require a large amount of infrastructure and is easily accessible.

1.2 Aims and Objectives

The aim of this project is to create an Android app, that doesn't require a network connection to function. The app takes in a text message, converts it to audio tones that represent the message, and plays them out of the speaker. This audio will then be picked up by a low-power handheld radio that transmits it as radio waves. The app will also do the opposite: taking in audio from a radio that is then converted back to text and displayed to the user. Another aim is to provide an option for encrypting messages, enabling secure communication between users. This functionality will then be tested under a range of conditions. Figure 1.1 shows a diagram of the app concept, outlining how a message is transmitted.

The objectives of the project are as follows:

1. Learn handheld radio operation and understand the science underpinning radio technology.
2. Obtain an RSGB Foundation Licence for amateur radio operation of 25 Watt (W) radios on a variety of bands [48].

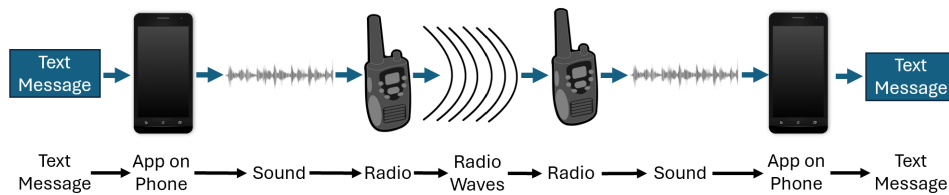


Figure 1.1: The message transmission pipeline. Images from: [51].

3. Develop a user-friendly interface that allows smooth operation of the application.
4. Design and implement an efficient text-to-audio conversion algorithm that encodes characters into a sequence of distinct audio frequencies for reliable transmission.
5. Integrate audio output functionality to ensure audio is reliably picked up by handheld radios.
6. Incorporate audio-to-text decoding to accurately interpret received audio, in different noisy scenarios.
7. Decide on an appropriate encryption algorithm and implement an optional encryption feature within the application to support secure message transmission.
8. Test transmission reliability under different environmental conditions, including varying distances, and background noise levels.

1.3 Background Concepts

1.3.1 Handheld Radios

A *handheld radio*, also known as a walkie-talkie, is a portable radio transceiver; this means that it can both transmit and receive radio waves. A typical handheld radio, hereinafter referred to as a *radio*, has a microphone, speaker, antenna, and a push-to-talk button (PTT).

The radio is normally in receive mode, where the antenna captures incoming radio frequency (RF) signals, demodulates them to retrieve an audio signal, amplifies this signal, and plays it through the speaker. When the PTT button is pressed, the radio

switches from receiving to transmitting mode. In this mode, the user's voice is captured by the microphone, converted into an electrical signal, and modulated onto a carrier wave; this signal is then amplified and broadcast via the antenna [9]. Figure 1.2 shows a diagram of the makeup of a radio transmitter.

Most radios also have a Voice Operated Exchange (VOX) setting which allows them to transmit when the input is above a certain adjustable volume, instead of requiring the user to hold the PTT button to transmit.

Modulation is a core concept in radio communication, referring to the process of varying a property of the carrier wave, such as its amplitude, frequency, or phase, in proportion to the information signal. This combines the signal from the microphone with a carrier, turning it into a radio wave containing the message that can be broadcast as an RF signal [33]. Radios most commonly use *Frequency Modulation (FM)* because it offers good noise immunity and is relatively simple to implement [56]. In FM, the frequency of the carrier wave is varied in direct proportion to the amplitude of the original information signal. As shown in Figure 1.3, when the input signal's amplitude increases, the carrier frequency shifts upward, and when the input signal's amplitude decreases, the carrier frequency shifts downward.

Radio communication is inherently constrained by several factors, including line-of-sight limitations, interference from other RF sources, multipath propagation, and regulatory transmission power limits [9]. In practice, most consumer handheld radios have an effective range of 1–5 miles under ideal conditions, though this can be significantly reduced in obstructed environments like cities [53].

1.3.1.1 Radio Licensing

Most countries regulate the use of the radio spectrum through licensing frameworks to ensure efficient allocation of frequencies and to prevent harmful interference between users. National regulatory authorities, such as the Federal Communications Commission (FCC) in the United States or the Office of Communications (Ofcom) in the United Kingdom, typically require individuals or organisations to obtain a licence to transmit on certain frequency bands or at higher power levels [20, 49]. Licence conditions often specify permissible frequency ranges, maximum transmission power, and approved equipment standards. Unlicensed use is generally restricted to designated bands, such as the Industrial, Scientific, and Medical (ISM) bands, and limited to low-power devices, such as Wi-Fi routers or handheld walkie-talkies, under strict technical parameters [36]. Furthermore, in the United Kingdom it is illegal to transmit encrypted

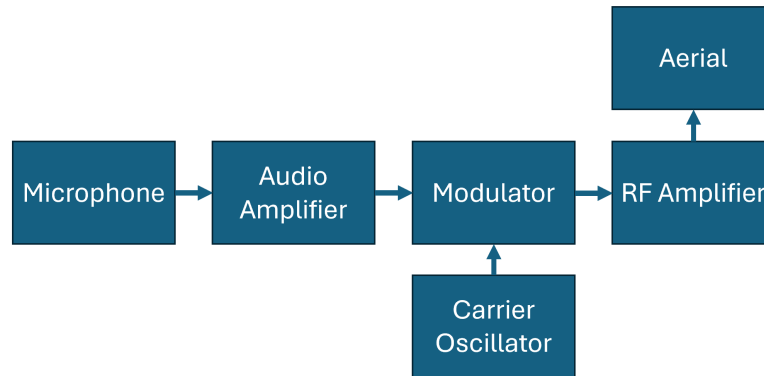


Figure 1.2: Block diagram of the internal components of a radio transmitter. Audio is received from the microphone and modulated with a high frequency carrier wave. The resulting RF signal is then transmitted via the aerial.

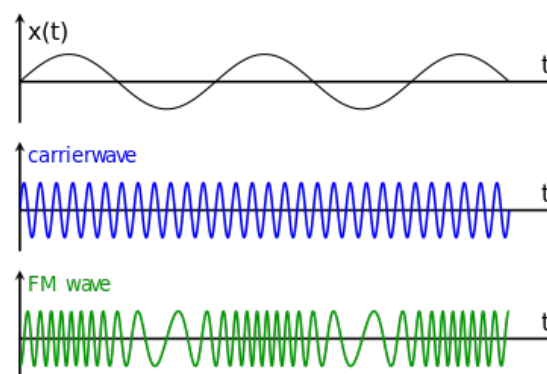


Figure 1.3: The effect of Frequency Modulation on a carrier wave. Source: [39].

information over licensed amateur radio bands. This means that testing of encrypted messages must be carefully planned, using alternatives to ensure compliance with legal and regulatory requirements.

As part of this project, the RSGB Foundation Licence (issued by Ofcom) was obtained which permits operation of radios with output power up to 25 W [48, 49]. This qualification enabled legal testing of the application under realistic deployment conditions, at appropriate power levels and in environments representative of actual hand-held radio use.

1.3.2 Fast Fourier Transform

The *Fast Fourier Transform* (FFT) is an efficient algorithm for computing the *Discrete Fourier Transform* (DFT) and its inverse. The DFT converts a sequence of discrete time-domain samples into a representation in the frequency domain, revealing the signal's constituent frequency components [68]. In the context of audio signals, the DFT allows us to transform the amplitude sampled in the time domain, into a comparison of amplitude against frequency, as shown in Figure 1.4.

The forward DFT is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \dots, N-1 \quad (1.1)$$

where $j = \sqrt{-1}$ denotes the imaginary unit, and $e^{-j\frac{2\pi}{N}kn}$ are the complex exponential basis functions. One can think of these basis functions as complex-valued waveforms that act as templates, where the DFT is calculating how much of each template

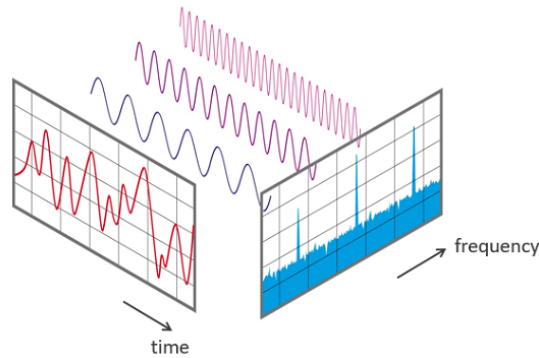


Figure 1.4: The effect of the DFT (and FFT). A view of a signal in the time and frequency domain. Source: [46].

to combine to create the original signal.

The inverse DFT reconstructs the original time-domain sequence from its frequency-domain representation:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j\frac{2\pi}{N}kn}, \quad n = 0, 1, \dots, N-1 \quad (1.2)$$

Computing the DFT directly from its definition requires $O(N^2)$ complex multiplications and additions, which becomes computationally expensive for large N . The FFT reduces this complexity to $O(N \log N)$ by exploiting the periodicity and symmetry of the complex exponential terms [11].

The most common FFT algorithm, the *Cooley–Tukey radix-2 decimation-in-time* method [11], assumes N is a power of two and recursively decomposes the N -point DFT into two $N/2$ -point DFTs: one for the even-indexed samples $x[2n]$ and one for the odd-indexed samples $x[2n+1]$. This decomposition can be expressed as:

$$X[k] = E[k] + W_N^k O[k], \quad X[k + N/2] = E[k] - W_N^k O[k] \quad (1.3)$$

where $E[k]$ and $O[k]$ are the DFTs of the even and odd parts respectively, and:

$$W_N^k = e^{-j\frac{2\pi}{N}k} \quad (1.4)$$

are the so-called “twiddle factors” [34]. On the complex plane a twiddle factor represents a rotation by $-\frac{2\pi k}{N}$ radians; they essentially act as the “glue” that combines the even and odd DFTs to form the full DFT. This divide-and-conquer approach dramatically reduces computation time, making real-time spectral analysis feasible in modern communication systems [64].

However, FFTs operate on discrete, finite-length sequences, which implies certain limitations. Frequency resolution is inversely proportional to the sampling window length, and leakage effects can occur if the signal period does not align with the analysis window. Windowing functions such as Hamming or Hann are therefore often applied before the FFT to reduce spectral leakage [32].

1.3.2.1 Frequency Modulation and Audio Signal Encoding

For transmitting digital information via audio, such as application-generated tones, the baseband data can be mapped to discrete audio frequencies using schemes like

frequency-shift keying (FSK), where binary symbols correspond to distinct tone frequencies that modulate a carrier wave [41]. At the receiver, the FFT allows these frequencies to be identified and the digital data reconstructed even in the presence of background noise [43]. Accurate analysis with the FFT requires proper digitization of the audio, which is governed by the Nyquist-Shannon sampling theorem: a signal must be sampled at least twice its highest frequency to avoid aliasing [60]. Thus, if the maximum transmitted tone frequency is f_{max} , the audio must be sampled at $2f_{max}$ or higher to ensure all components are captured. When transmitted over radio channels, these encoded audio signals can be carried using FM, linking the spectral analysis provided by the FFT with the practical robustness of FM transmission [33]. Together, FSK, FM, and FFT link the theoretical and practical aspects of audio-based digital communication, enabling reliable encoding, transmission, and decoding of information.

1.3.3 Cyber Security Principles

In cyber security, three core principles: *confidentiality*, *integrity*, and *availability*, form the foundation for designing and evaluating secure systems. These principles, often referred to as the *CIA triad*, are widely recognised in both academic and industry practice [59].

Confidentiality is the principle that restricts access to sensitive information, ensuring it is disclosed only to individuals or entities with legitimate authorisation. It further encompasses the right of individuals to exercise control over their personal data, including decisions regarding what information is collected and stored, and by whom it can be accessed or received [66].

Integrity refers to two related concepts. *Data* integrity ensures that information can only be modified through explicit authorisation and in accordance with established policies. *System* integrity ensures that no unauthorised manipulation can interrupt a system from performing its intended function fully [66].

Availability ensures that systems operate promptly and that authorised users are not denied access to services [66].

While these principles are broadly applicable across digital communication systems, their implementation in radio-based communication presents unique challenges. The open nature of radio signals allows transmissions to propagate to any receiver within range, making it difficult to guarantee confidentiality without encryption or other protective measures. Similarly, interference, environmental noise, and potential

malicious tampering can compromise the integrity of transmitted messages. Meanwhile equipment limitations, signal jamming, or spectrum congestion can threaten the availability. These vulnerabilities highlight that, in their basic form, radio systems cannot fully satisfy the CIA triad, leaving sensitive and confidential communications exposed to risk [44]. This gap underscores the importance of exploring methods to encode, transmit, and decode messages securely and reliably, thereby motivating this project to enhance the security and robustness of radio-based communication systems.

1.3.4 Mobile Phone Development

Mobile application development is dominated by two platforms: *Android* (Google) and *iOS* (Apple), which together account for over 99% of the global smartphone market [67]. Each provides its own development environment: Android applications are typically built in *Java* or *Kotlin* using Android Studio, while iOS applications use *Swift* or *Objective-C* in Xcode [3, 25].

Cross-platform frameworks such as React Native, Flutter, and Xamarin allow a single codebase to be deployed across both platforms, reducing development effort [65]. However, these tools can introduce overhead, restrict access to low-level features, and perform poorly in applications requiring precise timing and real-time processing [50].

This project adopts native Android development for two main reasons. First, Android's larger global market share, especially in regions where affordable devices dominate, makes it more accessible [67]. Second, native development provides full access to low-level audio Application Programming Interfaces (APIs), such as `AudioTrack`, enabling accurate, low-latency signal generation essential for synchronised radio transmission. Practical considerations, including the availability of Android devices for development and testing, further supported this choice.

Thus, the use of native Android ensures both broad accessibility and the technical performance required for secure, real-time radio communication.

1.3.5 Android App Structure

In the Android application framework, the primary building block of user interaction is the `Activity`. An `Activity` represents a single, focused task that the user can perform, typically corresponding to a distinct screen or interface within the application. By structuring an application into multiple activities, developers can modularise functionality, promoting separation of concerns and improving maintainability.

Each `Activity` comprises two fundamental components. First, a *layout resource*; this specifies the spatial arrangement, appearance, and properties of the user interface (UI) elements. Second, an associated *logic class*; this contains the programmatic control flow and event handling logic. This class dictates the behaviour of the activity at different lifecycle stages, such as initial creation, user interaction, and eventual destruction [25].

Activities operate within a well-defined lifecycle managed by the Android operating system. This includes callback methods such as `onCreate()`, `onStart()`, `onResume()`, `onPause()`, and `onDestroy()`, which enable developers to initialise resources, manage state transitions, and release resources when no longer needed. The lifecycle design supports responsiveness and efficient resource usage, which are critical for mobile environments with constrained processors, memory, and battery capacity [37]. This is particularly important in projects that interact with hardware components, external data sources, or that require real-time processing, as it ensures efficient use of system resources and consistent application behaviour across device states.

As discussed in Section 1.3.3, more functionality is needed to make radio communication secure. The flexibility of Android apps, compared to the fixed functions of radios, enables features that directly support confidentiality, integrity, and availability. In this context, real-time performance is critical; delays can undermine both data integrity and service availability. Android's lifecycle and threading model allow secure features to integrate with hardware efficiently, balancing background processing with a responsive interface. This ensures timely handling of sensitive data while maintaining a smooth user experience. By leveraging these capabilities, the proposed system can improve the security of radio communication without hardware modifications, making it adaptable and resilient to common threats.

1.4 Ethical Considerations

This project does not involve human subjects or personal information, sensitive or otherwise. The University of Manchester's Ethics Approval Tool [70] was used to confirm that no ethical approval is required (see Figure 1.5).

Outcome: Evaluation

1	Personal information?	No
2	Sensitive/confidential?	No
3	Vulnerable groups?	No
4	Risk of disclosures?	No

Ethical approval not required ⓘ

Based on the information you have provided, it does not appear that your project requires formal ethical approval. Before continuing however, please ensure you read the guidance in the information bubble above.

If you are a student, you **must** verify this outcome with your supervisor before starting your project.

Any queries should be directed to your supervisor or the [RCEIT Support Officer](#).

Please print a copy of this outcome for your records.

Figure 1.5: Confirmation from the University of Manchester Ethical Approval Tool.

Chapter 2

Methodology

This section outlines how the application was implemented. The app was named “Nuntius” from the Latin word for messenger.

2.1 Android App Framework

The Android app for this project was developed in *Java* [4], with *Extensible Markup Language (XML)* [45] used for layout resources, and built using *Gradle* [28].

Java is a platform-independent language widely adopted across domains [54]. Its strengths include scalability, efficient multi-threading, and Just-In-Time (JIT) compilation for near-native performance. The extensive ecosystem provides mature libraries, such as TarsosDSP, which were crucial for the signal-processing requirements of this project. Although Kotlin is the officially recommended Android `Activity` language, Java was chosen for its compatibility with legacy APIs and broad library support, ensuring robust, low-latency performance across diverse Android devices.

XML defines the app’s activity *layouts*, separating UI design from logic. This human-readable format allows rapid iteration and reuse of components while Android Studio provides real-time previews, streamlining design. Defining layouts directly in Java was considered but rejected in favour of XML’s flexibility and maintainability.

Gradle is the build automation system used by Android Studio. It handles dependency management, incremental builds, and build variants (e.g. debug vs. release) with higher efficiency than predecessors like Ant or Maven [28]. For this project, Gradle simplified library integration and enabled distinct development and evaluation builds, streamlining testing and deployment.

2.2 Sending Messages

The sending process converts a text message into an audio signal suitable for transmission over a handheld radio. This process consists of two main stages: Text-to-Audio Encoding, where characters are mapped to frequency-based chords, and Audio Playback, where the generated audio is output in real time. By separating these stages, the pipeline ensures both accurate representation of the underlying data and precise control over timing and signal quality.

2.2.1 Text-to-Audio Encoding

To transmit a text message over a handheld radio, the message must first be converted into an audio signal that represents the underlying data. In this project, each character is encoded into its binary representation and then mapped to one of a set of six pre-defined frequencies. The resulting sequence of tones is combined to form a unique chord that can be played through the device's speaker and transmitted via the radio. Six bits were chosen, as this allows sixty-three unique chords which is enough to represent A–Z, a–z, 0–9 and SPACE. More bits were considered, but this increases the complexity and requires smaller spaces between the frequencies which would reduce the robustness of frequency detection.

In addition to the data chords that correspond to characters, two special chords are defined and inserted at the beginning and end of the sequence, to indicate activation and deactivation of a transmission. A further special chord is inserted into longer messages for timing synchronisation (see Section 2.3.1).

This approach follows the principle of FSK (as mentioned in Section 1.3.2.1), in which discrete frequencies are used to represent digital symbols [43].

Several modulation schemes were considered, including phase-shift keying and chirp-based encoding. However FSK was chosen as it is simple to implement and known to perform well on low-bandwidth, low-power radio channels commonly used by handheld devices [30]. Furthermore, the use of fixed, pre-assigned tones allows the receiving application to detect the transmitted information using the FFT.

This encoding balances complexity and reliability, suiting the constraints of real-time radio use.

2.2.2 Audio Playback

After encoding a message as a sequence of chords, we output them as an audio signal. Nuntius uses an `AudioTrack` object to play raw *Pulse-Code Modulation (PCM)* data. PCM is a standard method for digitally representing analogue audio signals, valued for its simplicity and fidelity [71]. In PCM, the continuous waveform is sampled at a fixed rate and each sample is quantised to a discrete numeric value. `AudioTrack` was selected over alternative playback methods such as `MediaPlayer` or `SoundPool` because it supports low-latency streaming playback of raw data, which is essential for real-time transmission.

Each tone in the composite signal is generated using the standard sine wave equation:

$$x(t) = \sin(2\pi ft) \quad (2.1)$$

Where f is the frequency in Hertz (Hz). In discrete form (i.e., for each created audio sample), the time variable t is replaced by the sample index i divided by the sampling rate f_s , giving:

$$x[i] = \sin(2\pi \frac{i \cdot f}{f_s}). \quad (2.2)$$

To generate a chord (i.e., a tone consisting of multiple frequencies), the amplitudes of the respective sine waves are summed for each sample index, giving:

$$x[i] = \frac{1}{N} \sum_{k=1}^N \sin(2\pi \frac{i \cdot f_k}{f_s}). \quad (2.3)$$

where N = number of frequencies in the chord, and f_k = frequency of the k -th tone.

This technique is widely used in tone synthesis and corresponds to the superposition principle in signal processing [66]; it enables the simultaneous transmission of multiple frequencies, which is central to the chord-based encoding used in this project.

Since the audio data is output in 16-bit PCM format, each calculated sample value (a signed integer in the range -32768 to +32767) is split into two bytes and added to a byte array in little-endian order; by design, Android employs little-endian ordering across all devices [26]. Once the full buffer of PCM data is generated, it is written to the `AudioTrack` object and played through the device's speaker.

This method enables the application to synthesise tone sequences in real-time, with minimal processing overhead, and provides precise control over both frequency and

timing, making it suitable for transmission via low-power handheld radios.

2.3 Receiving Messages

Once a message is transmitted, the receiving device must accurately convert the incoming audio into its corresponding text. This process consists of two tightly coupled stages: Timing Synchronisation and Audio-to-Text Decoding.

2.3.1 Timing Synchronisation

Accurate results from the FFT requires that the FFT windows align precisely with the played chords. This means that both devices must start timing simultaneously and maintain consistent timing throughout the transmission.

To initiate the timing, we use an “activation chord” which serves a different function than normal data chords. The sending device, hereinafter referred to as the “sender”, plays the activation chord for longer than a standard chord and has unique frequencies that do not overlap with the 6 frequencies used for encoding bits. The receiving device (the “receiver”) continuously uses an `AudioRecord` object to capture raw audio in 16-bit PCM format, storing it in a buffer.

While it is waiting for the activation chord, the receiver performs multiple FFTs on the minimum audio sample size, using a function `isChordPresent(chord)`. Unlike data decoding (see Section 2.3.2), this function only determines *whether* the given chord is present, instead of *which* frequencies are present; this makes detection faster and more reliable for a single chord. Once the activation chord ends, this moment is used as a consistent synchronisation point between devices.

After this moment, there is one second of silence, allowing the chord’s end to be clearly distinguished. Data chords, which represent characters, are then sent for 2 seconds each, balancing timing accuracy and transmission speed, with a small 0.25 second gap between them. This gap prevents overlap between chords and accommodates small hardware timing inconsistencies.

Thus the system operates with two parallel processes, on separate devices:

- The sender writes a 2 second chord to the `AudioTrack` object and waits for 2.25 seconds while the audio plays.
- The receiver accumulates audio samples for 2.25 seconds and then decodes them (see Section 2.3.2).

Numerous methods were considered to implement this 2.25-second timing:

- `Thread.sleep(timeInMilliseconds)`.
- `System.currentTimeMillis() - timeInMilliseconds`.
- `System.nanoTime() - timeInNanoSeconds`.
- `LockSupport.parkNanos(timeInNanoSeconds)`.
- Accumulate 2.25 seconds worth of audio samples based on the Sample Rate.

After testing each method extensively, the most reliable approach was determined to be:

- **Sender:** Write chord to `AudioTrack`, then wait using `LockSupport.parkNanos(timeInNanoSeconds)`.
- **Receiver:** Accumulate audio samples until `System.nanoTime() - lastChordEndTime >= timeInNanoSeconds` is true.

Both functions are highly accurate, with `LockSupport.parkNanos()` being specifically designed “for creating higher-level synchronization utilities” [52]. However, timing drift still occurs over multiple characters, causing FFT windows to overlap chord boundaries and potentially leading to misidentification or skipped characters.

To mitigate drift, we introduce a “re-sync chord” after every 15 data chords. This mechanism mirrors the activation chord logic: the receiver performs rapid FFTs with `isChordPresent(chord)`, and the end of the re-sync chord is used as a new synchronisation point. Data transmission then resumes with the 2.25-second cycle.

Testing showed that without re-sync chords, timing became unreliable after approximately 20 characters. The 15 character interval ensures that drift is corrected before it impacts decoding.

2.3.2 Audio-to-Text Decoding

Once consistent timing has been established using the activation chord, audio-to-text conversion proceeds on the 2.25-second intervals of data chords. For each chord, a segment from the centre of the audio buffer is selected to minimise boundary effects and reduce spectral leakage.

Prior to applying the FFT, a Hann window is applied to the segment to further suppress discontinuities at the edges [32]. A Hann window was chosen rather than a rectangular or Hamming window because it offers superior side-lobe attenuation and therefore produces more accurate frequency estimates in the presence of closely spaced tones [47].

The FFT is then performed and the amplitude at each frequency is analysed. The receiver first checks for the presence of the deactivation chord; if it is not detected, frequency detection is performed to determine which tones are active in the current time slot. A frequency is considered present if its amplitude exceeds a predefined threshold, chosen based on observed signal-to-noise ratios (SNRs) during testing. To account for minor frequency shifts caused by transmission such as distortion, a small tolerance is permitted around each predefined tone [40]. Alternative detection methods such as the Goertzel algorithm were considered, but the FFT was chosen because it can analyse all relevant tone frequencies simultaneously with lower computational overhead, which is essential for real-time performance on mobile hardware.

Detected frequencies are mapped back to their associated bits and stored in an array. Once the deactivation chord is detected, the accumulated bit sequence is converted from binary into its corresponding text representation and displayed to the user. Figure 2.1 shows the main page of Nuntius, used for sending and receiving messages. Figure 2.2 summarises the process for sending and receiving messages.

2.3.2.1 TarsosDSP

To implement the FFT analysis in practice, the project uses the *TarsosDSP* digital signal processing library for Android [63]. TarsosDSP provides a high-level audio processing framework and integrates an efficient FFT implementation based on the *JTransforms* library [55]. JTransforms uses the *Cooley–Tukey radix-2 decimation-in-time* algorithm, which is an FFT that reduces the computational complexity of the Discrete Fourier Transform from $O(N^2)$ to $O(N \log N)$ [11].

TarsosDSP was chosen over implementing a custom FFT or using the basic FFT utilities available in the Android SDK, because it offers optimised performance on mobile hardware and is widely used in real-time audio applications. The use of an optimised library helps to ensure that spectral analysis can be performed continuously with minimal latency, which is critical for reliable detection and decoding of the transmitted tones.

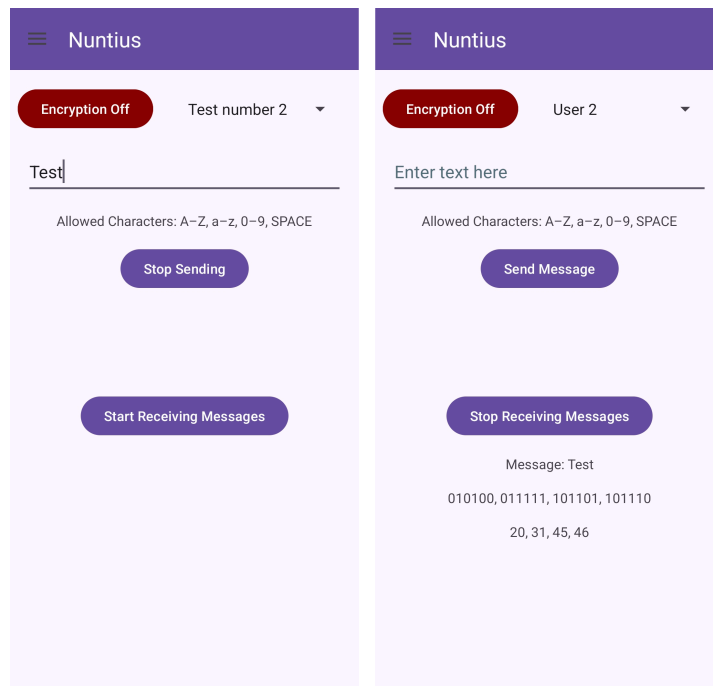


Figure 2.1: The sending and receiving messages page. Left shows a message being sent. Right shows a message having been received, with the binary and letter codes displayed underneath the message.

2.4 Encryption

Before sending a message, the user can choose whether to encrypt the message.

Android provides a large number of cipher transformations for encryptions, including AES, ARC4, BLOWFISH, ChaCha20, DES, DESede, and RSA. For this project, the *Advanced Encryption Standard (AES) in Galois/Counter Mode (GCM)* was selected due to its strong security guarantees, efficiency on modern hardware, and provision of authenticated encryption. AES is a *symmetric block cipher* standardised by NIST [12]. This means that the sender and receiver have to have a shared secret called a *key* which is used to encrypt and decrypt messages. This shared secret ensures that only the sender and receiver can obtain the original message. AES is widely recognised for its resistance to known cryptanalytic attacks when used with sufficiently long keys [10]. The recommended key lengths are 128, 192, and 256 bits [6]; we use 256-bit keys for maximum security.

GCM mode combines counter (CTR) mode encryption with a Galois field-based authentication tag, providing both confidentiality and integrity in a single operation.

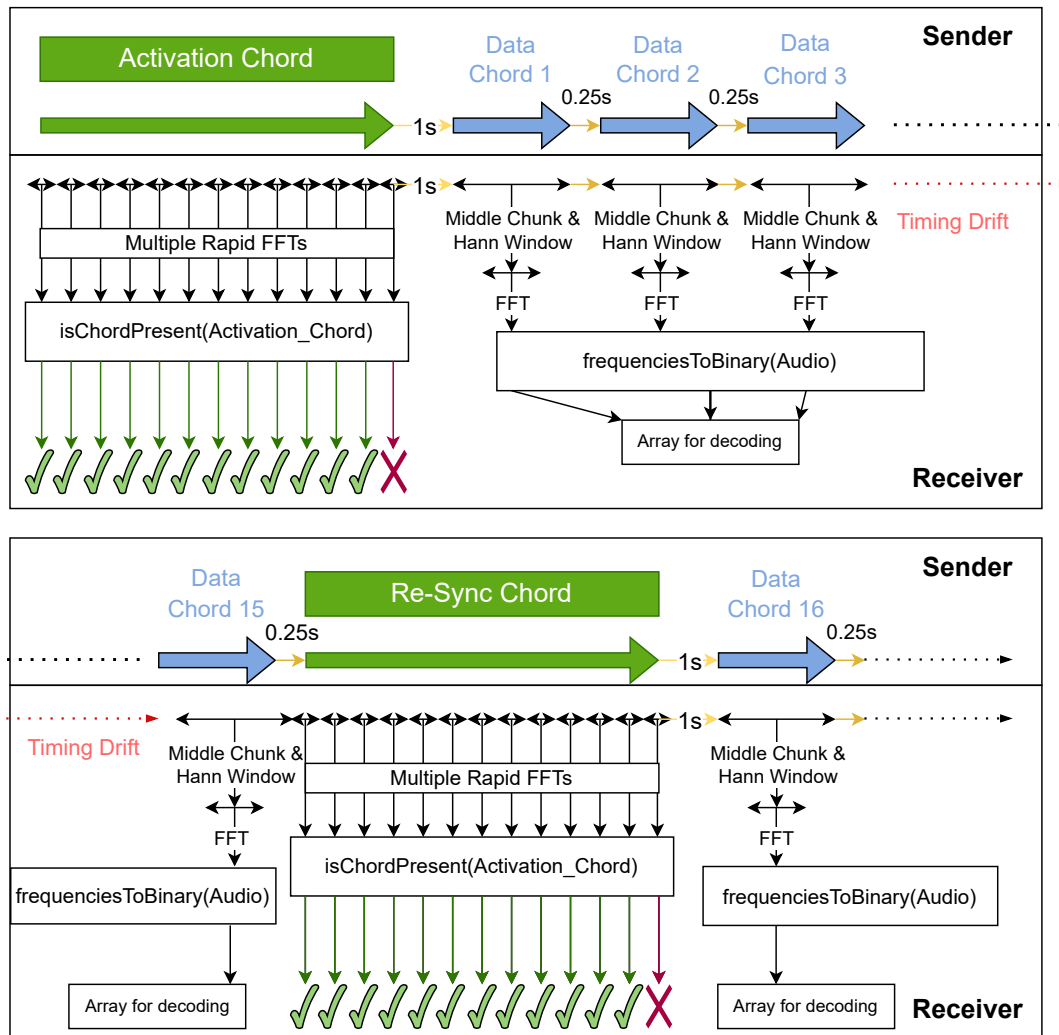


Figure 2.2: The timing mechanism between the sender and receiver. The top sender/receiver line directly precedes the bottom, connecting via the dotted line and red dotted arrow labelled "Timing Drift".

This makes it particularly suitable for message-based communication, where detecting tampering is as important as preventing unauthorised access. Each AES-GCM encrypted message is made up of: a header containing the Initial Value (IV) length, the IV, encrypted data, and the authentication tag, as illustrated in Figure 2.3. The authentication tag ensures that the message has not been tampered with – meaning it is the same as the one sent by the original sender. Also, the inclusion of the IV ensures that repeated encryption of identical plaintexts yields different ciphertexts, mitigating replay and pattern analysis attacks [17].

To transmit an encrypted message, the same procedure used for plaintext messages is applied (see Section 2.2). The message is divided into 6-bit segments, each of which is encoded as a chord for transmission. Upon reception, these chords are decoded to reconstruct the complete encrypted bit sequence. Finally, applying the AES-GCM decryption algorithm with the correct key recovers the original plaintext message.

Alternative modes, such as AES-CBC, offer confidentiality but require a separate message authentication code (MAC) to ensure integrity, increasing implementation complexity. Stream ciphers like ChaCha20, especially when paired with Poly1305 authentication [8], offer competitive performance and strong security. Conversely AES-GCM benefits from hardware acceleration on most modern Android devices via the ARMv8 Cryptography Extensions, resulting in lower latency and reduced power consumption [27].

Given the performance requirements of real-time message transmission in this project, along with the need for minimal overhead and robust protection against active and passive attacks, AES-GCM was the ideal choice.

An implementation decision concerned the ordering of encryption and encoding. One option was to first convert the plaintext message into a binary string and then

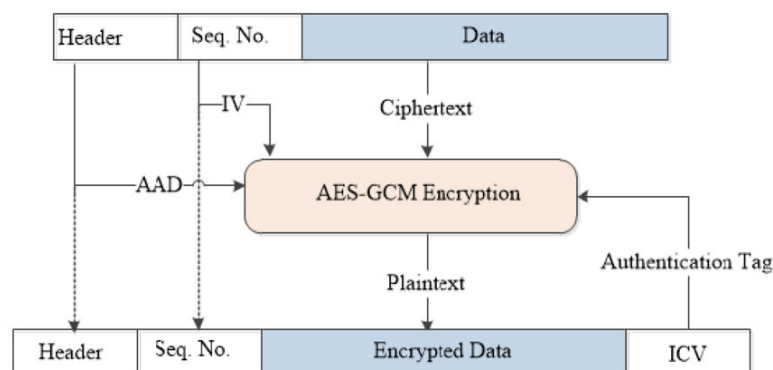


Figure 2.3: The AES-GCM decryption and authentication process. Source: [19].

encrypt, but this reduces plaintext entropy by mapping characters into predictable binary representations, increasing the risk of information leakage through ciphertext analysis [21]. Moreover, AES-GCM is designed for byte-level input, and operating on constructed binary strings can expose structural details such as message length or padding [44]. To avoid these issues, encryption was applied directly to the original message bytes, after which the resulting ciphertext was encoded into binary for tone generation. Because encoding occurs post-encryption, it does not weaken cryptographic strength or leak plaintext structure, making this the most secure and practical design choice.

2.4.1 Contacts

To manage the secret keys that are required for AES-GCM encryption, the contacts page allows users to store them separately for each person they want to securely communicate with. This is defined in a separate `Activity` to the sending and receiving page, creating a separation of functionalities that promotes good development practices.

This requires users to meet up in person and enter each other as a contact before they can use encrypted communication. The alternative to this is to use a key exchange protocol such as Diffie-Helman [15]. Whilst this allows users to communicate securely without first agreeing a key, on an open radio system it is impossible to verify the identity of a user without public-key infrastructure; otherwise we cannot authenticate that a public-key is from a certain user and the system becomes vulnerable to attacks [2]. Thus, a shared secret with no key-exchange protocols makes more sense for communication through radios, which would require large scale modification for this technology.

When users are agreeing on keys, one of them generates the key, whilst one of them enters it, both saving the entry in a new contact. The user entering the code can either type the key in manually or scan the QR code displayed on the other device. This is much faster and removes the risk of input errors, therefore streamlining the process.

The app generates valid, suitably long, AES-GCM keys and securely stores them in an `EncryptedSharedPreferences`. This prevents attackers that obtain a device from retrieving the stored secret keys without full access to the phone. Figure 2.4 shows the contacts page and process of entering a new entry.

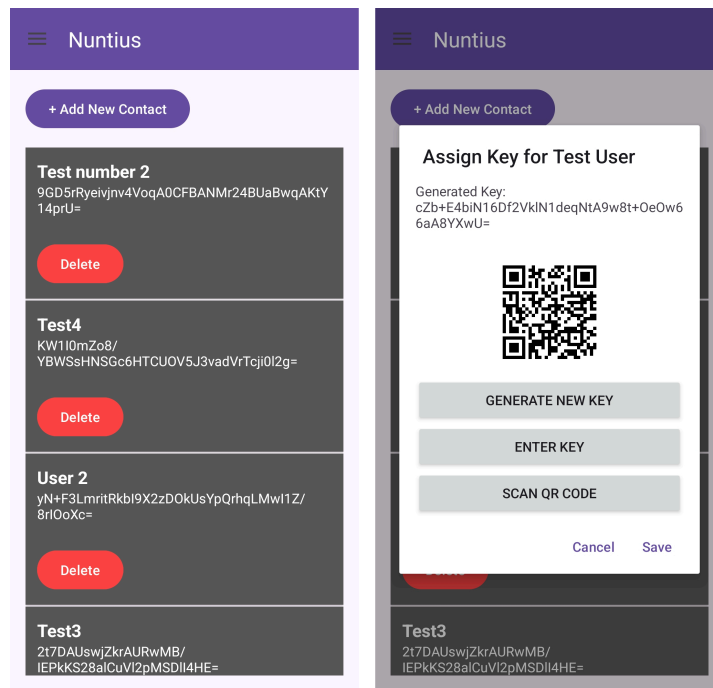


Figure 2.4: The left shows the contacts page, containing all saved contacts and their corresponding keys. The right shows the page for entering a new contact after the Generate New Key button has been pressed.

2.5 Handheld Radios

To evaluate the practical performance of the proposed system, a series of transmission and reception tests were conducted using a pair of *Baofeng UV-5R* handheld radios. The UV-5R is a widely available dual-band transceiver that operates in the 136 MHz to 174 MHz Very-High Frequency (VHF) and 400 MHz to 520 MHz Ultra-High Frequency (UHF) ranges and supports configurable transmission power levels up to 5 W [5]. It was selected for testing due to its low cost, ease of configuration, and its popularity among amateur radio users [5, 72].

To be able to operate these radios, the RSGB Foundation Licence (issued by Of-com [49]) was studied for and passed; this allows the user to transmit on radios with up to 20 W of power. Obtaining the licence was essential in order to legally conduct over-the-air transmission tests and evaluate the system under realistic conditions. Without it, testing would have been restricted to purely offline or laboratory scenarios, which would not accurately reflect real-world radio propagation effects such as interference, attenuation or multipath fading. The licence therefore enabled proper validation of Nuntius in the intended deployment environment.

Chapter 3

Evaluation and Reflection

3.1 Test Design

A series of structured experiments were designed to evaluate the performance of the application under a range of operating conditions. In order to ensure that all possible chords were exercised, and to test the effect of differing message lengths, a set of five messages were selected and transmitted. Table 3.1 shows the test messages 1–5.

Each test was performed with a Samsung S23 acting as the sender, and a Samsung S9 acting as the receiver; keeping these roles the same ensured consistency over all tests. Both these phones ran the standard version of Android (excluding Samsung’s custom One UI which doesn’t effect this project’s app). Additionally, Samsung is the most used Android phone manufacturer [42], demonstrating it as the appropriate choice for these tests.

Message	1	2	3	4	5
Message Text	A	BCDEFGH	IJKLMNOPQRSTU	VWXYZabc defghijklmn	opqrstuvwxyz012345 6789 ABCDEFGHI
Length	1	7	13	19	31
Re-Sync Chords	0	0	0	1	2
Length (E)	42	52	60	68	86
Re-Sync Chords (E)	2	3	4	4	5

Table 3.1: Messages, their lengths, and re-sync chords required. (E) denotes encrypted messages. (Line breaks are not a part of messages – they are for table spacing purposes only.)

It should be noted that, when encryption is enabled, the cipher text is longer than the plaintext, and therefore encrypted and non-encrypted messages are not directly comparable in terms of transmission length.

Each experiment was defined by three criteria that target different challenges to the functionality of Nuntius:

Transmission Range: The application was tested in three configurations: phone-to-phone (speaker to microphone), radio (close-range), and radio (long-range). The phone-to-phone configuration serves as a baseline, allowing performance to be measured without radio-specific impairments such as interference. The short and long range radio tests, in contrast, assess the system’s resilience when exposed to real RF propagation conditions. Close-range was approximately 10 metres, introducing radio operation but under good conditions. Long-range was approximately 0.5 miles (≈ 800 metres) in dense city conditions. A longer range, 1.5 mile test was attempted, but the *Baofeng UV-5R* radios couldn’t successfully communicate (with or without the app), most likely due to the high density of buildings and probable high interference.

Encryption State: In each range configuration, the tests were conducted with encryption enabled and disabled. This tests the effect of encryption on message receiving, evaluating whether the goal of encrypted messaging can be achieved without introducing unacceptable losses. Due to regulatory restrictions in the UK that prohibit the transmission of encrypted content over amateur radio bands, fully encrypted over-the-air tests could not be performed. Due to this limitation, a *faux-encryption* scheme was used for encryption tests done over radios (phone-to-phone was done using normal encryption). Plaintext messages were padded to match the length of their encrypted equivalents. By ensuring that every bit must be recovered for the message to be marked as received, we can provide an equivalent test of the impact of encryption on system performance.

Noise Profiles: In each scenario three noise profiles were tested against. These were: Quiet – no or minimal noise, Medium – a desk fan was on nearby, and Noisy – a looped portion of music was played. By including noise in the evaluation, we tested the robustness of the system to typical operating scenarios, which is essential for its practical usability. The quiet profile represented ideal operating conditions. A desk fan was chosen to represent medium noise, as it produces a consistent broadband hum that mimics common environmental interference in indoor settings without overwhelming the signal. The looped portion selected was the drop from *Contact* by *Daft Punk*, from 3 minutes 15 seconds to 4 minutes 48 seconds. This section was selected for its broad frequency content (see Figure 3.1), making it a realistic proxy for worst-case noisy environments.

The test matrix is summarised in Table 3.2. This test design directly aligns with

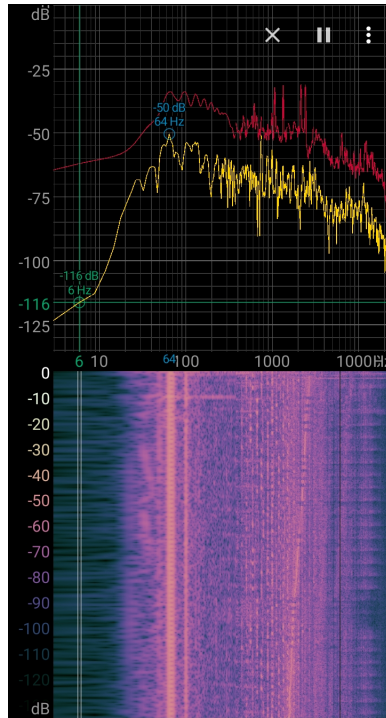


Figure 3.1: The frequencies present in the selected portion of *Contact* by *Daft Punk* [13]. Spectrum created using: [57].

the project’s objectives. Transmission range addresses Objective 8 (testing under varying conditions), encryption state addresses Objective 7 (ensuring secure communication without unacceptable loss), and noise profiles address Objective 6 (robustness of audio-to-text decoding). Under each unique test condition, messages 1 to 5 were sent and the success of the receiver was recorded (see Section 3.2 for more detail). By systematically varying these conditions, we can critically evaluate the system’s reliability, security, and usability in realistic scenarios.

Due to limited testing time, repeats of each test condition could not be performed. Additionally, being in a busy city environment, no quiet or medium noise tests could be performed as the baseline background noise was at a high level.

By combining different noise profiles, transmission ranges, encryption states, and message lengths, the test design provides a comprehensive evaluation of both the reliability and the robustness of the proposed system in distinct operating conditions.

Range	Encryption	Noise Profile
Phone to Phone	No	Quiet
		Medium
		Noisy
	Yes	Quiet
		Medium
		Noisy
Radio (Close)	No	Quiet
		Medium
		Noisy
	Faux	Quiet
		Medium
		Noisy
Radio (Long)	No	Quiet
		Medium
		Noisy
	Faux	Quiet
		Medium
		Noisy

Table 3.2: The test conditions.

3.2 Test Results

For the purposes of evaluation, received messages were scored according to their completeness and accuracy. A score of 1 was assigned when a message was received in its entirety without error, while a score of 0 was assigned when no intelligible characters were recovered or when all characters were received incorrectly.

In the case of unencrypted messages, intermediate scores between 0 and 1 were possible. These were calculated using:

$$Score = \frac{Correct\ Characters}{Total\ Characters}. \quad (3.1)$$

This approach reflects the fact that, for unencrypted transmissions, partial recovery of a message may still yield meaningful information to the user. It also provides a more granular measure of the application's performance across different test conditions.

By contrast, encrypted transmissions required perfect recovery of all transmitted bits, since any deviation would result in a corrupted or invalid message. As such, encrypted message trials were strictly binary: either successfully received (1) or entirely unsuccessful (0).

Table 3.3 shows the results for all test criteria combinations.

Table 3.4 shows the totals over all tests. The 58.90% success rate shows definite feasibility of the system, and the following sections discuss how the different factors contributed to this total.

Range	Encryption	Noise Profile	Message					Totals
			1	2	3	4	5	
Phone to Phone	No	Quiet	1	1	1	1	1	5.00
		Medium	1	1	1	0.95	1	4.95
		Noisy	1	1	1	0.95	0.87	4.82
	Yes	Quiet	1	1	1	1	1	5.00
		Medium	1	1	0	1	1	4.00
		Noisy	0	0	0	0	0	0.00
Close	No	Quiet	1	1	1	1	0.94	4.94
		Medium	1	1	1	0.95	0.90	4.85
		Noisy	1	0.71	1	0.89	0.26	3.87
	Faux	Quiet	1	1	0	0	0	2.00
		Medium	0	0	0	0	0	0.00
		Noisy	0	0	0	0	0	0.00
Long	No	Quiet	N/A	N/A	N/A	N/A	N/A	N/A
		Medium	N/A	N/A	N/A	N/A	N/A	N/A
		Noisy	1	0.5	0.31	0	0	1.81
	Faux	Quiet	N/A	N/A	N/A	N/A	N/A	0.00
		Medium	N/A	N/A	N/A	N/A	N/A	0.00
		Noisy	0	0	0	0	0	0.00

Table 3.3: Message success rates across ranges, encryption modes, noise profiles, and total messages received.

Total Messages Received	Total Messages Sent	Total Success Rate
41.23	70.00	58.90%

Table 3.4: Total messages sent and received, and success rate over all tests.

3.2.1 Message Length and Encryption

For interpretation of the effect of message lengths on success rate, two perspectives are important: plaintext length (the user's message) and transmitted length (after optional encryption expansion). The plaintext perspective reflects usability, while transmitted length explains the technical impact on the channel

As shown in Table 3.5, the success rate decreases as message length increases from one to thirty-one characters. Here we compare based on plaintext message lengths, since they represent the amount of message data being transmitted. As expected, the success rate decreases as message length increases from one to thirty-one characters.

This is because longer messages involve more characters to decode, increasing the chance of at least one frequency being incorrectly detected. They also increase the likelihood that the FFT windows will drift out of sync with the sent chords. Although the re-sync chords exist to counteract this effect, and clearly work due to longer messages still being correctly received, the presence of this mechanism creates another point-of-failure for the system, reducing the success rate.

Figure 3.2 illustrates this downward trend. Although the sample size is small and there is some variation, the fitted trend line confirms that longer messages consistently reduce success rate.

Message	(Plaintext) Message Length	Re-Sync Chords	Total /10	Success Rate
1	1	0	10.00	100.00%
2	7	0	9.21	92.14%
3	13	1	7.31	73.08%
4	19	1	7.74	77.39%
5	31	2	5.97	59.68%

Table 3.5: Message lengths (before encryption is applied) and success rates. Each message was transmitted seven times. Re-sync chords indicate how many times resynchronisation was performed during decoding.

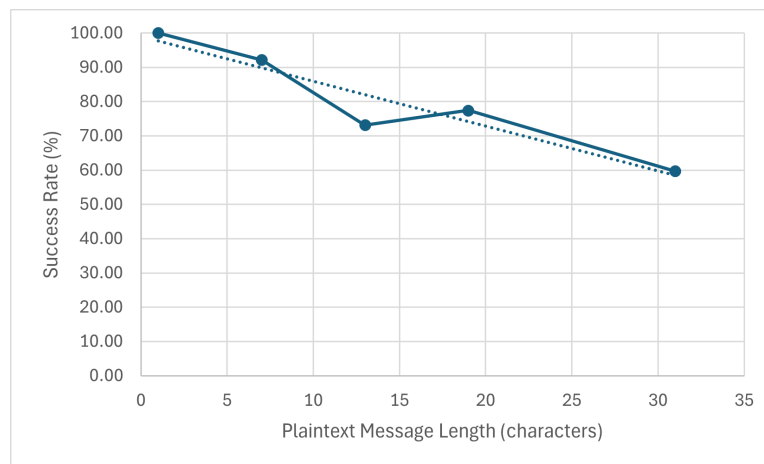


Figure 3.2: How success rate decreases as plaintext message length increases. The dotted line represents the trend line.

If we compare messages based on the length transmitted, this trend continues. Table 3.6 shows the total messages received for each message of increasing length.

A particularly sharp decrease is visible when moving from unencrypted message five (31 characters, 70.97%) to encrypted message one (42 characters, 42.86%), a significant 28.11% drop. From a user's perspective, this means that the same 31-character

message has a 71% success rate unencrypted, but only 27% when encrypted. This is explained by the stricter requirement that all bits must be correctly recovered for a message to be received. Even a single bit error results in complete message failure, which also explains why the trend for encrypted messages is less linear than for unencrypted ones. Table 3.7 supports this reasoning, showing that encryption causes an overall 54.97% decrease in success rate across all message lengths.

Figure 3.3 further highlights this effect by directly comparing unencrypted and encrypted performance. The visualisation makes clear that encryption not only increases the transmitted length, but also introduces an additional reliability penalty due to the all-or-nothing requirement.

Despite these reductions, Table 3.8 demonstrates that under ideal conditions (phone-to-phone, minimal/no noise) all messages were received, regardless of length or encryption. This confirms that the system is fundamentally capable of transmitting encrypted messages and therefore that Objective 7 (implement optional encryption) has been achieved. Thirty-one characters represents a practical message length, and encryption is essential for secure communication. Consequently, the challenge is not in the encryption mechanism itself, but in robustness under noisy or radio-based conditions. Future improvements should therefore focus on enhancing performance under these conditions, through error correction or improved synchronisation, rather than restricting message length or altering the encryption approach.

The impact of message length and encryption also interacts with other factors such as noise and range, which are explored in the following subsections.

3.2.2 Noise

Table 3.9 shows the received message scores for tests under the three noise conditions. Since quiet and medium noise conditions were not tested at long range, direct comparisons between noise levels use only the short- and mid-range data; long-range high-noise tests are reported for completeness, but excluded from the main noise-level analysis as no comparable baseline exists.

It is clear that the background noise has an effect on the ability to receive messages, with tests done under the quiet condition being the most successful, and high-noise tests being the least successful. The difference of 49.77% between quiet and high-noise tests show that this effect is significant. The most likely cause of this performance degradation is the way the receiver's FFT processes background noise. First, if the noise contains strong components near one of the designated bit frequencies,

Message	Message Length	Re-Sync Chords	Total /7	Success Rate
1	1	0	7.00	100.00%
2	7	0	6.21	88.77%
3	13	1	6.31	90.11%
4	19	1	5.74	81.99%
5	31	2	4.97	70.97%
1 (E)	42	2	3.00	42.86%
2 (E)	52	3	3.00	42.86%
3 (E)	60	4	1.00	14.29%
4 (E)	68	4	2.00	26.57%
5 (E)	86	5	2.00	26.57%

Table 3.6: Message lengths and success rates for unencrypted and encrypted messages. Each message was transmitted seven times. Lengths 1–31 correspond to messages 1 to 5 without encryption. Lengths 42–86 correspond to encrypted (or faux encrypted) messages (E) 1 to 5. Re-sync chords indicate how many times resynchronisation was performed during decoding.

Encryption	Total /35	Success Rate
No	30.23	86.40%
Yes/Faux	11.00	31.43%

Table 3.7: Success rates for encrypted and unencrypted messages across all tests (35 transmissions per group).

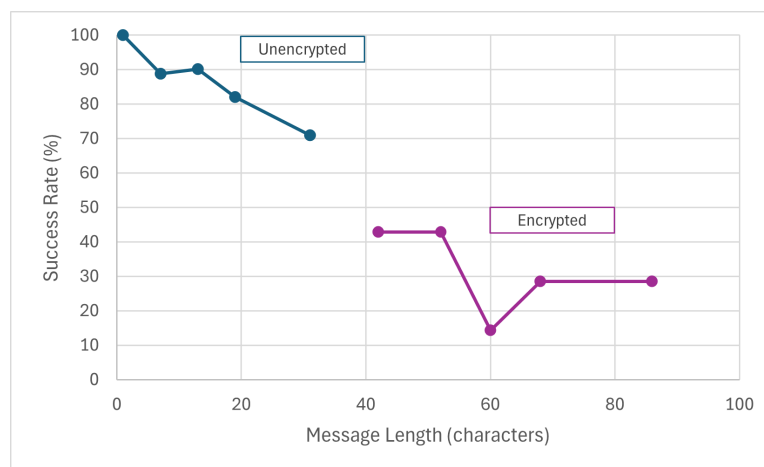


Figure 3.3: Success rate as message length increases and encryption is applied.

Encryption	Total /5	Success Rate
No	5	100.00%
Yes/Faux	5	100.00%

Table 3.8: Success rates for encrypted and unencrypted messages under ideal conditions (direct phone-to-phone transmission with minimal noise). All messages were received correctly, demonstrating the system’s full potential capability.

the FFT may incorrectly register that frequency as present, leading to false positives in symbol detection. Second, the addition of wideband or multi-tone noise raises the noise floor and reduces effective frequency resolution, making it more difficult for the system to distinguish between activation, re-synchronisation, or deactivation chords, and ordinary signal components. Together, these effects explain the lower success rates observed under noisy conditions.

Noise particularly affects encrypted messages (see Table 3.10). While unencrypted messages under noisy conditions have an 86.88% success rate, encrypted messages under the same condition have a 0.00% success rate. This occurs because a single bit error renders the entire encrypted message unrecoverable, exacerbating the impact of noise.

These findings highlight that while the system can operate under noise, its robustness is limited, particularly for encrypted transmissions. Objective 6 has been partially achieved but future work is needed for greater robustness. Such work could explore error correction mechanisms, such as forward error correction (FEC) codes [31, 62], adaptive modulation techniques [33], or averaging FFTs using a moving window. Such refinements would improve the resilience of the system in noisy environments and directly support the project’s objectives of enabling secure and reliable communication without hardware modifications.

Noise	Total (Exc. LR) /20	Total(Inc. LR)	Success Rate (Exc. LR)	Success Rate (Inc. LR)
Quiet	16.94	16.94 (/20)	84.68%	84.68%
Medium	13.80	13.84 (/20)	68.99%	68.99%
Noisy	8.69	10.50 (/30)	34.91%	28.96%

Table 3.9: Noise levels and total messages received. The second and fourth columns exclude long range (Exc. LR) as no quiet or medium noise tests were done for this level. The third and fifth columns include the long range test (Inc. LR) done at a noisy level.

Noise	Total (UE) /10	Total(E) /10	Success Rate (UE)	Success Rate (E)
Quiet	9.94	7.00	99.35%	70.00%
Medium	9.80	4.00	97.98%	40.00%
Noisy	8.69	0.00	86.88%	0.00%

Table 3.10: Noise levels and total messages received; messages split into unencrypted (UE) and encrypted (E).

3.2.3 Range

Table 3.11 shows the total messages received and success rate at each range. The limited tests at radio (long range) means we cannot directly compare ranges under all the possible conditions. Although we have this limitation, it is apparent that as range increases, successful message transmission decreases. Furthermore, if we directly compare ranges using the tests only done at noisy level (see Table 3.12) it supports this conclusion. However this suggests the effect of range is not as large as first suggested.

This was an expected effect of range attributable to several factors. By propagating a message using the radios, there is always some data loss stemming from signal loss and interference from other RF sources. As distance increases, received signal strength decreases following the free-space path loss model [1] (derived from the Friis transmission formula [22]), where power reduces proportionally to the square of the distance (due to the inverse-square law [58]). In practice, environments such as urban or indoor settings introduce additional attenuation through multipath fading, diffraction, and shadowing, further degrading the SNR [9, 33]. As seen in Section 3.2.2, a lower SNR increases the likelihood of bit detection errors, which is consistent with the observed drop in success rates at longer ranges. Interestingly, when comparing results under noisy-only conditions (Table 3.12), the relative penalty of distance appears smaller, suggesting that in these environments the dominant limiting factor is the noise itself rather than propagation loss.

Additionally, handheld radios often operate with limited transmit power and non-directional antennas, meaning range is inherently constrained compared to fixed or high-gain systems [33]. This explains why long-range results (≈ 800 metres) fell significantly below short-range performance. In the context of this project, this limitation is important; while short-range communication achieved moderate reliability even under noise, the degradation at long range highlights that the system is currently better suited to localised scenarios rather than wide-area communication.

To fully achieve Objectives 5, 6, and 8 (reliable audio with radios, in different noisy scenarios, and refined using testing) more work is required. This includes testing with

other, more powerful or directional radios. These further tests should also encompass more ranges and outside conditions; for example testing in rural areas where the level of interference is relatively low. Furthermore, as the range reduces the SNR, the work mentioned in Section 3.2.2 is relevant to improving the system’s success rate at range.

Range	Total	Success Rate
Phone-to-phone	23.77 (/30)	79.23%
Radio (Close Range)	15.65 (/30)	52.18%
Radio (Long Range)	1.81 (/10)	18.08%

Table 3.11: Ranges and total messages received. Close Range is approximately 10 metres and Long Range is approximately 800 metres (0.5 miles).

Range	Total /10	Success Rate
Phone-to-phone	4.82	48.21%
Radio (Close Range)	3.87	38.67%
Radio (Long Range)	1.81	18.08%

Table 3.12: Ranges and total messages received *under noisy conditions only*. Close Range is approximately 10 metres and Long Range is approximately 800 metres (0.5 miles).

3.3 Critical Reflection Outside Data

This section aims to evaluate the objectives that cannot be quantified within the above structured experiments.

Obtaining the RSGB Foundation Licence facilitated the aforementioned testing on amateur radio bands. This also furthered the project by fostering a greater understanding of the workings and science of radios.

The UI is simple and effective, ideal for a messaging system. Users are prompted with explanatory messages and any invalid operation is prevented and explained. For example, allowed characters are clearly listed below the input box, messages without this are stopped before sending, and an Android “Toast” message (temporary pop-up) explains this halt. Furthermore, the inclusion of a QR code to input keys makes for much quicker and more reliable operation than manual input, whilst still giving the option to type them in. However, this area could still be improved in the future, with a more polished UI being desirable, and more enticing for users unfamiliar with the system.

Chapter 4

Conclusion

4.1 Summary

This project set out to design a secure communication system that operates independently of mobile phone infrastructure. The motivation stemmed from the growing global emphasis on privacy, and the comparative advantage of radio, which unlike mobile networks, requires no centralised infrastructure. These motivations were addressed through the development of an offline Android application, capable of encoding text into audio signals, transmitting them over handheld radios, and decoding the received signals back into text. A further goal was to integrate strong encryption and to evaluate the system's performance under varied operating conditions.

To achieve this, radio operation was studied and an RSGB Foundation Licence obtained, enabling legal testing on amateur bands. An Android application “Nuntius” was implemented with a user-friendly interface and a custom algorithm, based on the principles of FSK, to convert text into raw PCM audio in real time. A complementary decoding process was developed to recover text from the received signal, with mechanisms to maintain synchronisation. AES-GCM encryption was successfully integrated, ensuring confidentiality, integrity, and authenticity of messages.

Testing demonstrated that the system performs reliably in favourable conditions, achieving an 84.66% success rate in quiet environments. However, its robustness decreases under adverse conditions, particularly in the presence of significant background noise, where performance dropped to 28.96%. These findings confirm the viability of the proposed system while highlighting areas for refinement, particularly in noise resilience and error correction. Opportunities for improvement and directions for future work are discussed in the following sections.

4.2 Limitations and Further Work

Although many of the project goals were achieved, some limitations were identified. The aforementioned resilience in the presence of noise, UI, and testing (including conditions, scope, and hardware) could all benefit from further work.

4.2.1 UI

While the current UI enables smooth operation and provides clear feedback during message transmission and reception, it remains relatively minimal. For broader adoption, greater emphasis on usability and accessibility would be beneficial. An introductory guide explaining system functionality, strategies to maximise transmission success, and the use of encryption would improve the experience for new or infrequent users.

Additional visual refinements, such as the inclusion of icons or subtle animations, could enhance engagement without compromising the simplicity that underpins the system's effectiveness. Future iterations should therefore focus on balancing clarity and polish, ensuring that the interface remains intuitive while appealing to a wider user base.

4.2.2 Testing

The 70 tests, with different ranges, encryption settings, message length, and noise levels, provided a valuable insight into the app's performance, and how different factors contribute to this. However, the testing framework was constrained in scope, and further experimentation is required to obtain more reliable and generalisable results.

4.2.2.1 Test Repetition

Due to limited time, only a single test was performed for each unique combination of conditions. As a result, some outcomes diverged from expected trends; for example Message 3 had a lower success rate than the longer Message 4 (see Table 3.2 and Figure 3.2). Repeating tests under identical conditions and reporting the mean success rate would provide a more robust measure of system performance. This approach would also align the evaluation with established principles of experimental reliability and reproducibility [7].

4.2.2.2 Conditions

The testing environment could also be expanded. In particular, quiet and medium-noise conditions were not evaluated for long-range transmissions, limiting direct comparisons across noise profiles. Furthermore, all long-range tests were conducted in a busy urban environment, where interference is high. Additional tests in rural or low-interference settings would provide a more representative indication of the system's potential in contexts where mobile infrastructure is limited, as highlighted in Section 1.1. Extended range testing could also help to establish the maximum viable communication distance under different environmental conditions.

4.2.2.3 Hardware

Hardware selection further constrained the evaluation. While the *Baofeng UV-5R* was an appropriate test platform due to its affordability and widespread use, future work should assess performance across a broader range of radios to capture variability in transmission and reception quality.

Similarly, tests were limited to two Samsung devices, which are broadly representative of mainstream Android hardware. Further evaluation across different smartphones would provide insight into how factors such as processing speed, microphone sensitivity, and speaker fidelity affect performance. Importantly, testing on lower-cost or older devices would help determine the system's suitability for deployment in resource-constrained regions, one of the target demographics for this project.

4.2.3 Noise Resilience

As discussed in Sections 3.2.2 and 3.2.3, a lower SNR ratio was observed to substantially reduce message success rates. Noise can be introduced through a variety of mechanisms, including environmental background noise, RF interference, multipath fading, diffraction, and shadowing. Improving resilience to these effects would significantly enhance the system's robustness in real-world communication scenarios. Two particularly promising approaches are forward error correction (FEC) and adaptive modulation.

4.2.3.1 Forward Error Correction (FEC)

Forward Error Correction (FEC) provides a mechanism by which the sender encodes the message so that the receiver can not only detect but also correct a limited number of errors. Although this increases forward channel bandwidth, it often reduces the overall cost of communication by avoiding retransmissions.

A naïve form of FEC is repetition coding, where the message is transmitted multiple times and the receiver takes a majority vote on each bit. However, this approach is inefficient in both time and bandwidth, and still prone to repeated errors. More sophisticated methods, such as Turbo codes, offer far greater resilience.

Turbo codes, among the most powerful modern error-correction techniques, enable systems to operate close to the Shannon capacity limit [61] (the theoretical maximum). They employ two encoders: one processes the input bit stream directly, while the other processes an interleaved (scrambled) version of the same stream. The outputs are then transmitted alongside the original data. At the receiver, two iterative decoders exchange reliability estimates of each bit, gradually converging towards the correct sequence [29].

While computationally complex, this method is feasible in the proposed system, since encoding and decoding occur outside the strict timing requirements of chord-based transmission. While message length increases with the addition of parity data, this drawback may be outweighed by the improved success rates in noisy environments. A hybrid approach could also be explored, applying error correction only to longer or high-priority messages, while shorter messages remain uncoded to preserve efficiency.

4.2.3.2 Adaptive Modulation

Another avenue for improving resilience is adaptive modulation, which dynamically adjusts the modulation scheme to suit current channel conditions. In favourable environments (e.g., high SNR), higher-order modulation can increase data throughput, while in poor conditions the system can fall back to simpler, more robust schemes [24, 41].

Implementing adaptive modulation in this project would require two key components. First, the receiver must perform *channel estimation*, measuring indicators such as SNR or bit error rate (BER) to infer current channel quality. Second, a *feedback and adjustment mechanism* would synchronise the sender and receiver when modulation

depth changes. This could be achieved through a short control message sent prior to transmission, or by adopting a fixed schedule based on environmental cues, such as operating in quiet versus noisy profiles.

4.2.3.3 Signal Averaging FFT with Moving Window

Noise resilience could also be improved by applying a moving-window FFT across the 2 s tone interval. Instead of a single FFT, multiple overlapping windows would be analysed and their spectra averaged, reinforcing true frequency peaks while suppressing random noise. This averaging improves the SNR by a factor of $\sqrt{n}$, where n is the number of windows [14].

The technique requires little extra computation, integrates cleanly with the current decoding pipeline, and could significantly sharpen tone detection under noisy conditions without increasing bandwidth or transmission time.

Collectively, FEC, adaptive modulation, and signal-averaging FFT represent complementary avenues for enhancing noise resilience. FEC improves reliability through redundancy, adaptive modulation optimises efficiency by aligning transmission parameters with channel conditions, and FFT averaging strengthens tone detection by reducing stochastic noise. Together, these methods could significantly advance the system's robustness and applicability in diverse real-world communication scenarios.

Bibliography

- [1] IEEE standard for definitions of terms for antennas. *IEEE Std 145-2013 (Revision of IEEE Std 145-1993)*, pages 1–50, 2014.
- [2] M. Alatawi and N. Saxena. SoK: An analysis of end-to-end encryption and authentication ceremonies in secure messaging systems. In *IEEE Transactions on Information Theory*, volume 22 of *WiSec '23*, page 187–201, New York, NY, USA, 2023. Association for Computing Machinery.
- [3] Apple. Xcode. <https://developer.apple.com/xcode/>. [Online; accessed 22-August-2025].
- [4] K. Arnold, J. Gosling, and D. Holmes. *The Java programming language*. Addison Wesley Professional, 2005.
- [5] Baofeng. Baofeng UV-5R. <https://www.baofengradio.co.uk/baofeng-uv-5r-dual-band-uhf-vhf-ham-radio-two-way-fm-walkie-talkie-earpiece/>. [Online; accessed 17-August-2025].
- [6] E. Barker and A. Roginsky. Transitioning the use of cryptographic algorithms and key lengths. NIST Special Publication 800-131A Revision 2, National Institute of Standards and Technology, 2019.
- [7] A. Berchtold. Test-retest: Agreement or reliability? *Methodological Innovations*, 9:1–7, 2016.
- [8] D. Bernstein. ChaCha, a variant of Salsa20. *The University of Illinois at Chicago*, 2008.
- [9] A. Betts. *The Foundation Licence Manual for Radio Amateurs*. Radio Society of Great Britain (RSGB), 2024.

- [10] X. Bonnetain, M. Naya-Plasencia, and A. Schrottenloher. Quantum security analysis of aes. *IACR Transactions on Symmetric Cryptology*, 2019(2):55–93, 2019.
- [11] J. W. Cooley and J. W. Tukey. An Algorithm for the Machine Calculation of Complex Fourier Series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [12] J. Daemen and V. Rijmen. *The Design of Rijndael: AES – The Advanced Encryption Standard*. Information Security and Cryptography. Springer Berlin, Heidelberg, 1st edition, 2002.
- [13] Daft Punk. Contact. Album: Random Access Memories, 2013. Track 13.
- [14] J. Dempster. Chapter six - signal analysis and measurement. In *The Laboratory Computer*, Biological Techniques Series, pages 136–171. Academic Press, London, 2001.
- [15] W. Diffie and M. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.
- [16] S. Duguin and P. Pavlova. The role of cyber in the Russian war against Ukraine: Its impact and the consequences for the future of armed conflict. Technical report, The CyberPeace Institute, 2023. Requested by the European Parliament, subcommittee on Security and Defence (SEDE).
- [17] M. Dworkin. Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007.
- [18] M. Evans. *Just War Theory: A Reappraisal*. 2016.
- [19] A. Fadele, A. Jegede, and C. Eke. Open access robust data security framework for IoT network using integrated techniques. *International Journal of Applied Mathematics and Machine Learning*, 13:91, 2020.
- [20] Federal Communications Commission (FCC). Licensing Information. <https://www.fcc.gov/licensing>. [Online; accessed 14-August-2025].
- [21] N. Ferguson, B. Schneier, and T. Kohno. *Cryptography Engineering: Design Principles and Practical Applications*. Wiley, 2010.

- [22] H. Friis. A note on a simple transmission formula. *Proceedings of the IRE*, 34(5):254–256, 1946.
- [23] Global System for Mobile Communications Association (GSMA). The state of mobile internet connectivity report. <https://www.gsma.com/r/somic/>, 2024. [Online; accessed 10-August-2025].
- [24] A. Goldsmith and S.-G. Chua. Variable-rate variable-power mqam for fading channels. *IEEE Transactions on Communications*, 45(10):1218–1230, 1997.
- [25] Google. Android Developers. <https://developer.android.com/develop>. [Online; accessed 14-August-2025].
- [26] Google - Android Developers. Android ABIs - NDK. <https://developer.android.com/develop>, 2023. [Online; accessed 15-August-2025].
- [27] C. P. L. Gouvêa and J. López. Implementing GCM on ARMv8. In K. Nyberg, editor, *Topics in Cryptology — CT-RSA 2015*, pages 167–180, Cham, 2015. Springer International Publishing.
- [28] Gradle, Inc. Gradle Build Tool. <https://gradle.org/>. [Online; accessed 10-August-2025].
- [29] E. Guizzo. Closing in on the perfect code. *IEEE Spectrum*, Mar. 2004. Updated: 25 Feb 2025, [Online; accessed 27-August-2025].
- [30] M. C. Gursoy. Error rate analysis for peaky signaling over fading channels. *IEEE Transactions on Communications*, 57(9):2546–2550, 2009.
- [31] R. W. Hamming. Error detecting and error correcting codes. *The Bell System Technical Journal*, 29(2):147–160, 1950.
- [32] F. Harris. On the use of windows for harmonic analysis with the discrete Fourier transform. *Proceedings of the IEEE*, 66(1):51–83, 1978.
- [33] S. S. Haykin and M. Moher. *Communication Systems*. Wiley, 2010.
- [34] M. Heideman, D. Johnson, and C. Burrus. Gauss and the history of the fast Fourier transform. *Archive for History of Exact Sciences*, 34(4):265–277, 1985.

- [35] International Committee of the Red Cross (ICRC). Geneva Convention Relative to the Protection of Civilian Persons in Time of War (Fourth Geneva Convention). 1949.
- [36] International Telecommunication Union. Radio Regulations. <https://www.itu.int/pub/R-REG-RR>. [Online; accessed 14-August-2025].
- [37] K. Jakimoski and A. Andonoska. Performance Evaluation of Mobile Applications. In *Proceedings of the Third International Conference on Autonomic and Autonomous Systems (ICAS'07)*, Struga, Macedonia, 2018.
- [38] T. Jawaaid. Privacy vs national security. *International Journal of Computer Trends and Technology (IJCTT)*, 68(7), 2020.
- [39] Jitender Singh. Frequency modulation. <https://www.concepts-of-physics.com/modern/frequency-modulation.php>, 2023. [Online; accessed 16-August-2025].
- [40] S. Kay. *Fundamentals of Statistical Signal Processing: Detection theory*. Fundamentals of Statistical Signal Processing. Prentice-Hall PTR, 1998.
- [41] G. Kennedy and B. Davis. *Electronic Communication Systems*. Electrical Engineering Series. Glencoe, 1993.
- [42] N. Kumar. Android usage statistics 2025 – versions & global market share. <https://www.demandsage.com/android-statistics/>, 2025. [Online; accessed 28-August-2025].
- [43] B. P. Lathi and Z. Ding. *Modern Digital and Analog Communication Systems*. Oxford University Press, 5th edition, 2022.
- [44] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1st edition, 1997.
- [45] Mozilla. XML Documentation. <https://developer.mozilla.org/en-US/docs/Web/XML>. [Online; accessed 10-August-2025].
- [46] NTi Audio. Fast Fourier Transformation FFT - Basics. <https://www.nti-audio.com/en/support/know-how/fast-fourier-transform-fft>. [Online; accessed 13-August-2025].

- [47] A. Nuttall. Some windows with very good sidelobe behavior. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 29(1):84–91, 1981.
- [48] Office of Communications (Ofcom). Amateur radio guidance - licensing guidance document for amateur radio. https://www.ofcom.org.uk/siteassets/resources/documents/manage-your-licence/amateur/amateur_radio_licence_guidance_for_licensees.pdf?v=368087. [Online; accessed 22-August-2025].
- [49] Office of Communications (Ofcom). Radio Broadcast Licensing. <https://www.ofcom.org.uk/licences/radio-broadcast>. [Online; accessed 14-August-2025].
- [50] A. Olusola Olajide, A. Omotayo, O. Adebola, T. Omomule, and S. Orimoloye. 2018.
- [51] Open Clip Art Library. Open clip art. <https://openclipart.org/>, 2004. [Online; accessed 16-August-2025].
- [52] Oracle. Java™ platform, standard edition 8 API specification - Class Lock-Support. <https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/locks/LockSupport.html>. [Online; accessed 22-August-2025].
- [53] OS Comms. What you need to know about walkie talkie range. <https://oscommsonline.com/blogs/news/what-you-need-to-know-about-walkie-talkie-range>. [Online; accessed 12-August-2025].
- [54] G. Pinto, W. Torres, B. Fernandes, F. Castor, and R. S. Barros. A large-scale study on the usage of Java’s concurrent programming constructs. *Journal of Systems and Software*, 106:59–81, 2015.
- [55] Piotr Wendykier. Jtransforms. <https://github.com/wendykierp/JTransforms>, 2014. [Online; accessed 17-August-2025].
- [56] M. Plonus. *Electronics and Communications for Scientists and Engineers*. Academic Press, 2001.
- [57] C. Reinke. Spectroid. <https://play.google.com/store/apps/details?id=org.intoorbit.spectrum>, 2014. [Online; accessed 23-August-2025].
- [58] A. Ryer. *Light Measurement Handbook*. International Light, 1997.

- [59] D. Schatz, R. Bashroush, and J. Wall. Towards a More Representative Definition of Cyber Security. *Journal of Digital Forensics, Security and Law*, 12:53, 2017.
- [60] C. Shannon. Communication in the presence of noise. *Proceedings of the IRE*, 37(1):10–21, 1949.
- [61] C. E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.
- [62] L. Shu. *Error Control Coding: Fundamentals and Applications*. Pearson, 2004.
- [63] J. Six, O. Cornelis, and M. Leman. TarsosDSP, a Real-Time Audio Processing Framework in Java. In *Proceedings of the 53rd AES Conference (AES 53rd)*, 2014.
- [64] S. W. Smith. *The scientist and engineer's guide to digital signal processing*. California Technical Publishing, USA, 1997.
- [65] A. Sommer and S. Krusche. Evaluation of cross-platform frameworks for mobile applications. 03 2013.
- [66] W. Stallings. *Cryptography and Network Security: Principles and Practice*. Prentice Hall Press, USA, 6th edition, 2013.
- [67] Statcounter. Mobile operating system market share worldwide. <https://gs.statcounter.com/os-market-share/mobile/worldwide>. [Online; accessed 22-August-2025].
- [68] D. Sundararajan. *The Discrete Fourier Transform: Theory, Algorithms and Applications*. G - Reference, Information and Interdisciplinary Subjects Series. World Scientific, 2001.
- [69] United Nations. Universal declaration of human rights. 1948.
- [70] University of Manchester. Ethics Decision Tool. https://www.training.itservices.manchester.ac.uk/uom/ERM/ethics_decision_tool/story.html. [Online; accessed 12-August-2025].
- [71] J. Watkinson. *Art of Digital Audio*. Routledge, 3rd edition, 2001.

- [72] T. Witherspoon. The BaoFeng UV-5R is tougher than the \$25 price tag implies. <https://swling.com/blog/2015/10/the-baofeng-uv-5r-is-tougher-than-the-25-pricetag-implies/>, 2015. [Online; accessed 31-August-2025].

Appendix A

Software Versions

A.1 Android Studio

Software	Version
Android Studio	Android Studio Meerkat Feature Drop — 2024.3.2 Patch 1
Android SDK	Target: 35, Minimum: 24
Android Gradle Plugin	8.10.1
Java	1.8.0_431
Gradle	8.11.1

A.2 Build Dependencies

```
dependencies {  
    implementation(libs.core)  
    implementation(libs.appcompat)  
    implementation(libs.material)  
    implementation(libs.activity)  
    implementation(libs.constraintlayout)  
    implementation(libs.security.crypto)  
    implementation("com.google.zxing:core:3.5.1")  
    implementation("com.journeyapps:zxing-android-embedded:4.3.0")  
    testImplementation(libs.junit)  
    androidTestImplementation(libs.ext.junit)  
    androidTestImplementation(libs.espresso.core) }
```

Listing A.1: Gradle Dependencies

A.3 Phones used for testing

A.3.1 Sender

Attribute	Description
Device Model	Samsung S23
One UI	7.0
Android	15
RAM	8 GB
Storage	256 GB

A.3.2 Receiver

Attribute	Description
Device Model	Samsung Galaxy S9
One UI	2.5
Android	10
RAM	4 GB
Storage	64 GB

Appendix B

System Parameters

Description	Value
Activation Chord Frequencies	1070 Hz, 2150 Hz
Deactivation Chord Frequencies	1610 Hz, 2690 Hz
Re-Sync Chord Frequencies	1070 Hz, 2690 Hz
Bit Frequencies	800 Hz, 1340 Hz, 1880 Hz, 2420 Hz, 2960 Hz and 3500 Hz
Sample Rate	44 100 Hz (most stable on Android)
Android Permission Required	<code>Manifest.permission.RECORD_AUDIO</code>
Android Cipher Transformation	“AES/GCM/NoPadding”